فازبندی بک‌اند

پروژه درس مهندسی نرم‌افزار | فاز اول، اسپرینت اول | برنامه مدیریت وظایف (Todo App)

نام برنامه: Done & Dusted | **شعار:** Get it done and dusted

زبان‌ها: فارسی (پیش‌فرض) و انگلیسی | **تم:** لایت و دارک

نام ریپو و پکیج: `done-and-dusted` | بدون علامت و فاصله

نقش: توسعه‌دهنده بک‌اند: API، دیتابیس، ولیدیشن سمت سرور، کدهای خطا و پایداری داده

ابزارها: Express.js، TypeScript، Prisma، PostgreSQL، Zod، cors، dotenv

برنچ گیت: `backend` | **پوشه:** `backend/`

اصل کار موازی: در فاز ۰ قرارداد API با هم نوشته می‌شود. از فاز ۱ تا ۲ هر نفر مستقل کار می‌کند (با Postman، Thunder Client یا curl تست می‌کنید و منتظر فرانت نمی‌مانید). در فاز ۳ دو بخش به هم وصل می‌شوند. فازها مرحله‌ای هستند و به روز مشخصی وابسته نیستند (کل پروژه حدود ۷ تا ۱۰ روز).

فاز ۰: آماده‌سازی مشترک (با هم‌تیمی)

هدف: قبل از جدا شدن، همه چیزهای مشترک روشن شود تا بعدش کسی منتظر دیگری نماند.

- ☐ Product Backlog بنویسید (عنوان، توضیح، اولویت، معیار پذیرش) و آیتم‌های «پشتیبانی از دو زبان فارسی/انگلیسی» و «تم لایت/دارک» را هم داخلش بگذارید
- ☐ Sprint Backlog بسازید: هدف اسپرینت، آیتم‌های انتخابی، تسک‌های فنی، تقسیم کار
- ☐ Definition of Done را تعیین کنید (کد زده شده، تست دستی شده، روی برنچ push شده، معیار پذیرش برآورده شده)
- ☐ قرارداد API، کدهای خطا و محدودیت‌ها را نهایی کنید (جدول زیر)
- ☐ تصمیم‌های مشترک: نام برنامه Done & Dusted، نام ریپو `done-and-dusted`، تاریخ نسخه فارسی شمسی باشد یا میلادی
- ☐ ساختار پوشه‌ها، `.env.example`، `.gitignore` و README اولیه روی `main` و ساخت برنچ‌های `frontend` و `backend`
- ☐ جلسه هماهنگی اول را برگزار و ثبت کنید

قرارداد API (مرجع مشترک)

```
POST /api/tasks   body: { "title": string, "description"?: string }
                  201 -> Task
                  400 -> { "error": "<ERROR_CODE>" }
GET  /api/tasks   200 -> Task[]   (newest first)

Task   = { id, title, description, status, createdAt }
status = "TODO"      (the only value in sprint 1)
Limits = title: 1-100 chars after trim, description: up to 500 chars
```

کد خطا	HTTP	چه زمانی؟
`TITLE_REQUIRED`	۴۰۰	عنوان ارسال نشده، خالی یا فقط فاصله است
`TITLE_TOO_LONG`	۴۰۰	عنوان بیش از ۱۰۰ کاراکتر است
`INVALID_INPUT`	۴۰۰	نوع داده نادرست یا توضیحات بیش از ۵۰۰ کاراکتر یا JSON خراب
`SERVER_ERROR`	۵۰۰	خطای غیرمنتظره سرور

API مستقل از زبان است: سرور فقط کد خطا می‌فرستد و فرانت آن را به فارسی یا انگلیسی ترجمه می‌کند. محدودیت‌های ۱۰۰ و ۵۰۰ پیشنهادی است و باید با هم‌تیمی نهایی شود، چون هر دو طرف باید یکسان رعایتشان کنند.

خروجی / معیار تکمیل: Backlogها آماده، قرارداد API و کدهای خطا مکتوب، ریپو و برنچ‌ها آماده.

فاز ۱: راه‌اندازی زیرساخت

هدف: سرور Express و دیتابیس PostgreSQL بالا بیایند و مدل Task ساخته شود.

- ☐ ساخت پروژه Express با TypeScript داخل `backend/` (با `tsx` یا `ts-node`)
- ☐ نصب PostgreSQL (محلی یا Docker Compose) و ساخت دیتابیس (مثلاً `done_and_dusted`)
- ☐ نصب Prisma و تنظیم `.env` و `.env.example` با `PORT`، `DATABASE_URL` و `CORS_ORIGIN`
- ☐ مدل `Task` در `schema.prisma` و اجرای `npx prisma migrate dev`
- ☐ تنظیم `cors` (فقط آدرس فرانت، مثلاً `http://localhost:3000`)، `dotenv` و `express.json()`
- ☐ ساختار پوشه‌ها: `validators`، `controllers`، `routes`
- ☐ route سلامت `GET /health`

نمونه مدل Prisma

```
model Task {
  id          Int      @id @default(autoincrement())
  title       String
  description String?
  status      String   @default("TODO")
  createdAt   DateTime @default(now())
}
```

خروجی / معیار تکمیل: سرور با یک دستور بالا می‌آید، به دیتابیس وصل است و migration اجرا شده است.

فاز ۲: پیاده‌سازی قابلیت‌ها

هدف: دو endpoint اصلی، ولیدیشن و کدهای خطا کامل و با ابزار تست دستی بررسی شود.

- ☐ `POST /api/tasks`: دریافت `title` و trim، `description` کردن عنوان و ذخیره با پاسخ ۲۰۱
- ☐ `GET /api/tasks`: همه وظایف مرتب بر اساس `createdAt` نزولی
- ☐ ولیدیشن با Zod و محدودیت‌های قرارداد (عنوان ۱ تا ۱۰۰ کاراکتر، توضیحات حداکثر ۵۰۰)
- ☐ پاسخ خطا فقط شامل کد باشد: `{ "error": "TITLE_REQUIRED" }` و بدون متن فارسی یا انگلیسی
- ☐ ذخیره و بازگرداندن درست عنوان‌های فارسی و انگلیسی (یونیکد)
- ☐ وضعیت پیش‌فرض `TODO` و ثبت خودکار زمان ایجاد
- ☐ middleware خطا: JSON خراب ← `INVALID_INPUT` و خطای غیرمنتظره ← `SERVER_ERROR` با کد ۵۰۰
- ☐ تست دستی همه حالت‌های جدول زیر با Postman یا curl
- ☐ (اختیاری) چند تست خودکار با Vitest/Jest و Supertest

جدول تست دستی

ورودی	نتیجه مورد انتظار
`{}`	۴۰۰ و `TITLE_REQUIRED`
`{"title":" "}`	۴۰۰ و `TITLE_REQUIRED`
عنوان ۱۰۱ کاراکتری	۴۰۰ و `TITLE_TOO_LONG`
`{"title":123}`	۴۰۰ و `INVALID_INPUT`
توضیحات ۵۰۱ کاراکتری	۴۰۰ و `INVALID_INPUT`
عنوان فارسی و انگلیسی معتبر	۲۰۱ و ذخیره درست در دیتابیس

خروجی / معیار تکمیل: هر دو endpoint درست کار می‌کنند، خطاها فقط کد استاندارد دارند و داده در دیتابیس می‌ماند.

فاز ۳: اتصال و یکپارچه‌سازی (با هم‌تیمی)

هدف: فرانت و بک واقعاً به هم وصل شوند و همه سناریوها از ابتدا تا انتها کار کنند.

- ☐ اطمینان از اینکه CORS آدرس فرانت را قبول می‌کند
- ☐ هماهنگی با نفر فرانت: نام فیلدها، فرمت (ISO) `createdAt` و کدهای خطا
- ☐ رفع باگ‌هایی که موقع اتصال پیدا می‌شود
- ☐ سناریو ۱: ثبت وظیفه موفق ← در `GET` دیده می‌شود
- ☐ سناریو ۲: عنوان خالی یا بیش از ۱۰۰ کاراکتر ← ۴۰۰ با کد درست و عدم ثبت در دیتابیس

- [ ] سناریو ۳: ریستارت سرور ← داده‌ها باقی می‌مانند (Persistence)
- [ ] سناریو ۴: عنوان فارسی و انگلیسی درست ذخیره و برگردانده می‌شود
- [ ] مرج برنچ‌های `frontend` و `backend` به `main`

خروجی / معیار تکمیل: برنامه کامل روی `main` اجرا می‌شود و همه سناریوها درست کار می‌کنند.

فاز ۴: مستندسازی و تحویل (با هم‌تیمی)

هدف: همه تحویلی‌های اسپرینت آماده و بررسی شوند.

- [ ] بخش بک‌اند README: نصب PostgreSQL، تنظیم `.env`، `prisma migrate`، اجرای سرور و فهرست کدهای خطا
- [ ] مطمئن شوید `.env.example` کامل است و `.env` کامیت نشده
- [ ] ریپو را از صفر کلون کنید و طبق README اجرا کنید
- [ ] Sprint Review: قابلیت‌های کامل‌شده، میزان تحقق هدف، بازخوردها، پیشنهاد ادامه
- [ ] Sprint Retrospective با ۴ پرسش و حداقل یک اقدام اصلاحی مشخص و عملی
- [ ] جمع‌بندی گزارش ۴ جلسه
- [ ] مرور چک‌لیست ۸ تحویلی

خروجی / معیار تکمیل: هر ۸ مورد تحویلی در ریپو موجود است و پروژه از صفر قابل اجراست.

جلسات هماهنگی (۴ جلسه)

جلسه	زمان	موضوع
۱	پایان فاز ۰	تعیین قرارداد API، کدهای خطا، Backlogها و تقسیم کار
۲	پایان فاز ۱	بررسی راه‌اندازی سرور و دیتابیس، مشکلات نصب
۳	پایان یا وسط فاز ۲	پیشرفت endpointها، ولیدیشن و کدهای خطا، موانع قبل از اتصال
۴	پایان فاز ۳	نتیجه اتصال، باگ‌های باقی‌مانده، آماده‌سازی تحویل

هر جلسه ۱۵ تا ۲۰ دقیقه است و همان لحظه سه مورد ثبت می‌شود: پیشرفت نسبت به هدف، موانع، تصمیم‌ها و کارهای بعدی.

نکات مهم

- [ ] ولیدیشن سمت سرور الزامی است و به کلاینت اعتماد نکنید
- [ ] API مستقل از زبان است: فقط کد خطا بفرستید، نه متن
- [ ] `.env` را کامیت نکنید و `.env.example` را حتماً کامیت کنید
- [ ] هر تغییر در قرارداد، کدهای خطا یا محدودیت‌ها را فوراً به نفر فرانت اطلاع دهید
- [ ] قابلیت‌های اضافه (ویرایش، حذف، تغییر وضعیت) فقط در Product Backlog با اولویت پایین نوشته شود

دستورات گیت (برنچ شما)

```
git checkout backend
git pull origin backend
git add .
git commit -m "feat(backend): add create task api"
git push origin backend
```